Лекция №5

Хранение данных. Volumes и Bind Mounts

Постоянные данные Docker и первый контейнер PostgreSQL

Цели занятия

- Объяснить, почему данные в записываемом слое контейнера могут быть потеряны.
- Выбирать между Docker Volume и Bind Mount.
- Создавать, подключать и диагностировать именованные тома.
- Запускать PostgreSQL 17 с постоянным хранилищем.
- Проверять сохранность данных при пересоздании контейнера.

На что опираемся

- Лекция 1: образ и контейнер — разные объекты.
- Лекция 2: docker run, stop, rm, exec, logs и inspect.
- Лекция 3: получение и выбор образов.
- Лекция 4: Dockerfile для Express API; сегодня готовим отдельную базу данных для будущего подключения.

Учебные вопросы

№	Учебный вопрос
1	Файловая система контейнера и проблема сохранности данных
2	Docker Volumes: создание, подключение, управление
3	Bind Mounts: связь с каталогами хоста
4	PostgreSQL с постоянным хранилищем
5	Проверка сохранности, диагностика и безопасность

Почему данные контейнера могут исчезнуть

Записываемый слой контейнера и его жизненный цикл

Контейнер: неизменяемые слои и запись

- Образ формируется из read-only слоёв.
- При создании контейнера Docker предоставляет отдельный записываемый слой.
- Изменения внутри этого слоя принадлежат конкретному контейнеру.
- При удалении контейнера его записываемый слой удаляется.

Главная идея

Остановка контейнера не равна удалению, но удаление контейнера удаляет его записываемый слой.

Почему это важно для программиста

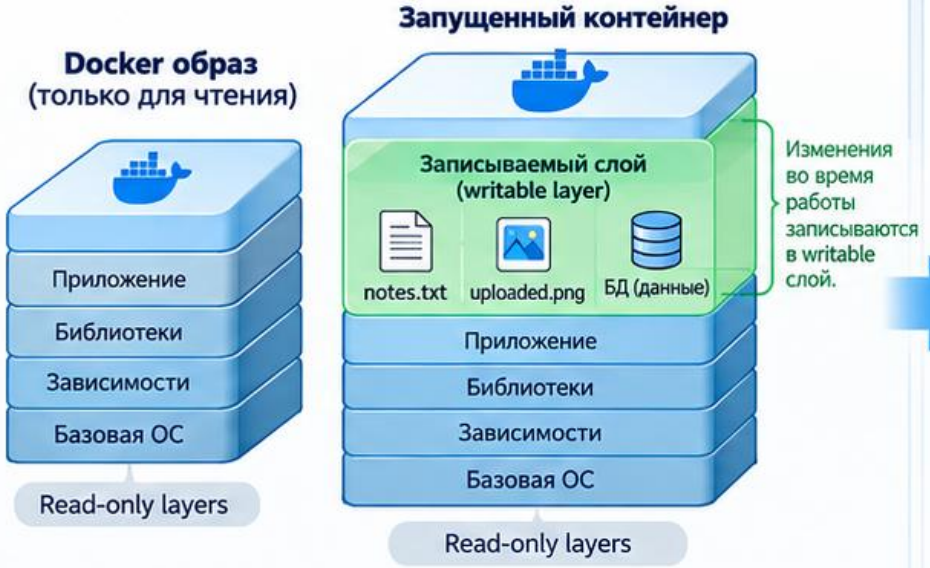
- Веб-приложение может загружать файлы и создавать отчёты.
- СУБД записывает таблицы и транзакционные журналы на диск.
- Пересоздание контейнера часто требуется при обновлении версии образа.
- Для долговременных данных следует подключать хранилище вне записываемого слоя контейнера.

Записываемый слой контейнера и потеря данных

Почему данные внутри контейнера могут исчезнуть после его удаления

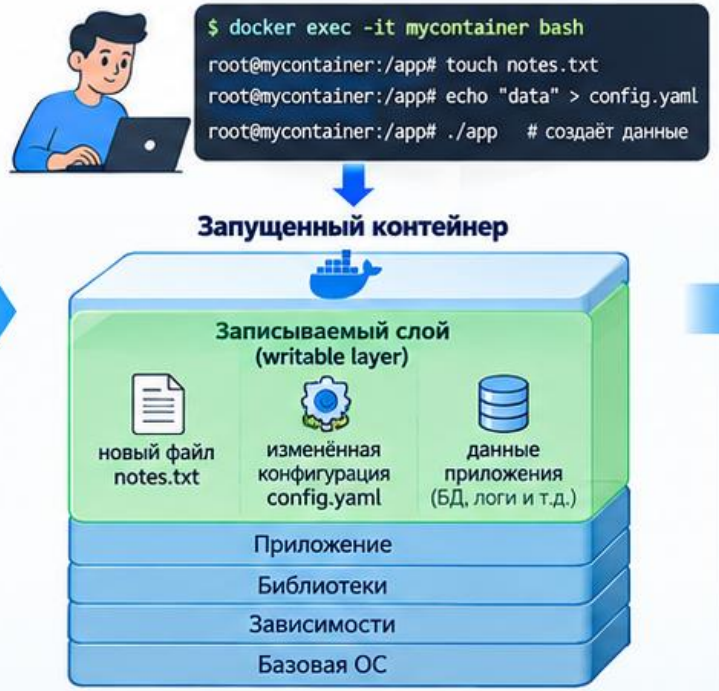
1 Образ и запущенный контейнер

Контейнер создаётся из образа и имеет дополнительный записываемый слой сверху.



2 Работа с контейнером

Пользователь работает с запущенным контейнером и создаёт в нём данные.



3 Удаление контейнера

При удалении контейнера его записываемый слой удаляется вместе с ним, и все данные в нём теряются.

```
$ docker rm mycontainer
```



Данные в записываемом слое не являются постоянными!

После удаления контейнера все файлы, изменения конфигурации и данные приложетеия теряются.

Без volume / bind mount



- данные записываются в writable layer
- после удаления контейнера данные теряются
- подходит только для временных данных

С volume или bind mount



- данные хранятся вне контейнера
- пересоздание контейнера не удаляет данные
- подходит для баз данных и пользовательских файлов

Что важно понять

1. Контейнер создаётся из образа.
2. Образ остаётся, даже если контейнер удалён.
3. Записываемый слой контейнера временный.
4. Для сохранения данных нужны volumes или bind mounts.

Что происходит без постоянного хранилища

1. **Шаг 1** — Создаём контейнер из образа.
2. **Шаг 2** — Приложение записывает данные внутрь записываемого слоя.
3. **Шаг 3** — Контейнер останавливается: данные пока остаются.
4. **Шаг 4** — Контейнер удаляется: его записываемый слой исчезает.
5. **Шаг 5** — Новый контейнер из того же образа не наследует эти изменения.

Остановить или удалить контейнер

docker stop

- Основной процесс останавливается.
- Контейнер остаётся в docker ps -a.
- Его записываемый слой остаётся.

docker rm

- Объект контейнера удаляется.
- Его записываемый слой удаляется.
- Именованный Docker Volume сохраняется, пока его не удалят отдельно.

Docker Volumes

Постоянное хранилище, которым управляет Docker

Определение Docker Volume

- Volume — управляемая Docker область хранения данных.
- Том существует независимо от жизненного цикла конкретного контейнера.
- Один и тот же именованный том можно повторно подключать к новым контейнерам.
- Docker предоставляет команды для создания, просмотра и удаления томов.

Docker Volume и жизненный цикл контейнера

Как данные сохраняются независимо от создания, остановки, удаления и пересоздания контейнера



Главная идея:

Контейнеры могут создаваться, останавливаться, удаляться и пересоздаваться.
Docker Volume продолжает существовать и хранить данные.

Без volume



Данные в writable layer контейнера

- Данные находятся в writable layer контейнера.
- После удаления контейнера данные **теряются**.
- Подходит только для временных данных.

С volume



Данные хранятся в отдельном volume (вне контейнера)

- Данные хранятся вне контейнера.
- После удаления контейнера данные **сохраняются**.
- Подходит для баз данных и пользовательских файлов.

Практический пример

Команда запуска:

```
docker run -d --name pg1 \
  -v pgdata:/var/lib/postgresql/data \
  postgres:17
```

1. Создаём записи в БД (например, таблицы и данные).
2. Удаляем контейнер: `docker rm pg1`.
3. Создаём новый контейнер с тем же volume.
4. Данные остались — можно работать дальше.

Что важно понять

1. Volume живёт дольше контейнера.
2. Удаление контейнера не удаляет volume автоматически.
3. Один и тот же volume можно снова подключить к новому контейнеру.
4. Для удаления volume нужна отдельная команда.

Команды работы с томами

Команда	Назначение
docker volume create mydata	Создать именованный том
docker volume ls	Просмотреть тома
docker volume inspect mydata	Исследовать метаданные
docker volume rm mydata	Удалить неиспользуемый том
docker volume prune	Удалить неиспользуемые тома: только с осознанной проверкой

Создание тома

BASH

```
sudo docker volume create pgdata-lab
```

Что важно

- Команда создаёт именованный том pgdata-lab.
- Повторный вызов с тем же именем не создаёт новый том.
- Проверка: docker volume ls показывает pgdata-lab.

Просмотр томов и метаданных

BASH

```
sudo docker volume ls
sudo docker volume inspect pgdata-lab
```

Что важно

- inspect покажет Driver и Mountpoint.
- Фактическое расположение зависит от Docker Engine и конфигурации хоста.
- Не изменяйте файлы тома вручную в каталоге Docker.

Монтирование тома через --mount

BASH

```
sudo docker run --rm \
  --mount type=volume,src=pgdata-lab,dst=/data \
  alpine:3.22 sh -c "echo persistent > /data/check.txt"
```

Что важно

- src — имя тома Docker; dst — путь внутри контейнера.
- --rm удалит временный контейнер после выполнения команды.
- Именованный том pgdata-lab остаётся после удаления контейнера.

Читаем файл из нового контейнера

BASH

```
sudo docker run --rm \
  --mount type=volume,src=pgdata-lab,dst=/data \
  alpine:3.22 cat /data/check.txt
```

Что важно

- Выполняется новый контейнер из того же образа.
- Ожидаемый результат: persistent.
- Это доказывает, что файл сохраняется вне жизненного цикла первого контейнера.

Именованный и анонимный том

Named Volume

- Явное имя: pgdata-lab.
- Проще повторно подключить к другому контейнеру.
- Удобен для учебной базы данных.

Anonymous Volume

- Не имеет удобного пользовательского имени.
- Может быть создан автоматически или явно.
- Сложнее отслеживать и повторно использовать.

Bind Mounts

Монтирование файла или каталога хоста в контейнер

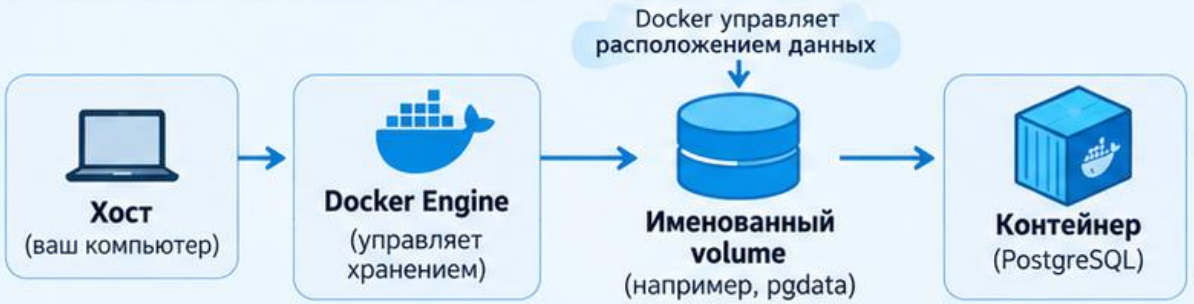
Определение Bind Mount

- Bind Mount — прямое подключение файла или каталога хостовой системы к пути внутри контейнера.
- В отличие от Volume, источник задаётся конкретным путём хоста.
- Изменение файлов может быть видно как на хосте, так и в контейнере.
- Полезно для разработки и передачи статических файлов или конфигурации.

Volume и Bind Mount — два способа хранения

Сравнение двух способов подключения данных в Docker

Docker Volume



- Данные хранятся в области, управляемой Docker
- Удобно для баз данных и постоянных данных
- Независим от структуры каталогов хоста
- Легко переносить и повторно подключать

Пример: PostgreSQL с постоянным volume

```
docker run -d --name pg1 \
  -v pgdata:/var/lib/postgresql/data \
  postgres:17
```

Bind Mount



- Подключает конкретную папку или файл хоста
- Удобно для исходного кода, конфигов и статических файлов
- Изменения на хосте сразу видны в контейнере
- Зависит от структуры каталогов и прав на хосте

Пример: сайт из локальной папки

```
docker run -d --name web1 \
  -v /home/user/site:/usr/share/nginx/html \
  nginx:stable
```

Краткое сравнение

Критерий	Docker Volume	Bind Mount
Кто управляет хранением	Docker	Хостовая ОС / пользователь
Типичный сценарий	Базы данных, постоянные данные	Код проекта, конфиги, статические файлы
Переносимость	Выше	Ниже
Прямая работа с файлами хоста	Ограничена	Да
Удобство в разработке	Среднее	Высокое

Итог: Volume чаще используют для постоянных данных, Bind Mount — для удобной разработки и доступа к файлам хоста.

Когда что выбирать

Docker Volume

- Данные PostgreSQL и других СУБД.
- Состояние приложения, независимое от структуры каталогов хоста.
- Управление средствами docker volume.

Bind Mount

- Исходный код при разработке.
- Конфигурационные файлы и тестовые HTML-страницы.
- Прямой обмен файлами с конкретным каталогом Debian.

Создаём каталог на хосте

```bash
mkdir -p "$HOME/docker-lab5/site"
printf "Docker bind mount works\n" > \
  "$HOME/docker-lab5/site/index.html"
```

Что важно

- Каталог создаётся в домашней директории текущего пользователя Debian.
- Файл index.html содержит строку для проверки.
- На хосте можно менять файл обычным редактором.

Монтируем каталог в Nginx

BASH

```
sudo docker run -d --name lab5-web -p 8085:80 \
  --mount type=bind,src="$HOME/docker-lab5/site"\
  ,dst=/usr/share/nginx/html,readonly nginx:stable
```

Что важно

- type=bind использует каталог Debian.
- readonly защищает подключённый каталог от записи из контейнера.
- Содержимое каталога хоста закрывает файлы, ранее находившиеся по целевому пути контейнера.

Проверяем bind mount

```bash
curl http://localhost:8085
sudo docker inspect lab5-web
```

Что важно

- В ответе ожидается Docker bind mount works.
- В docker inspect найдите Mounts и Type: bind.
- При изменении index.html на хосте ответ Nginx изменится без пересборки образа.

Осторожно с Bind Mount

- По умолчанию bind mount может предоставлять запись в файлы хоста.
- readonly снижает риск случайного изменения файла со стороны контейнера.
- Если путь источника отсутствует, поведение --mount и -v различается.
- Не монтируйте без необходимости чувствительные каталоги хоста.

PostgreSQL с постоянным хранилищем

Первый сервис базы данных в сквозном проекте

Зачем PostgreSQL отдельный контейнер

- Веб-приложение и база данных имеют разные задачи.
- На этой теме работаем только с PostgreSQL и его дисковыми данными.
- Связь Express API с PostgreSQL организуем на последующих занятиях.
- PostgreSQL должен переживать пересоздание контейнера без потери таблиц.

Выбор версии PostgreSQL и каталога данных

- Для лабораторной выбран официальный образ postgres:17.
- В PostgreSQL 17 и ниже постоянный том подключают к /var/lib/postgresql/data.
- Начиная с postgres:18 структура PGDATA и путь монтирования официального образа изменены.
- Не заменяйте 17 на 18 механически, оставляя прежний путь монтирования.

Главная идея

В примерах дальше используем postgres:17 и /var/lib/postgresql/data.

Контейнер PostgreSQL и постоянный Docker Volume

Данные базы хранятся в Docker Volume и сохраняются при пересоздании контейнера

1 Запуск контейнера PostgreSQL с томом

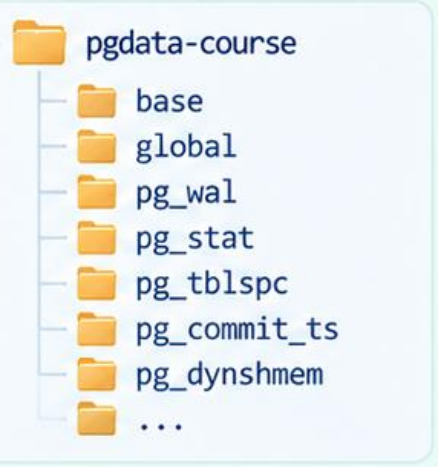
```
>_ docker volume create pgdata-course

   docker run -d \
     --name pg-course \
     -e POSTGRES_PASSWORD=secret \
     -p 5432:5432 \
     --mount source=pgdata-course,\
       target=/var/lib/postgresql/data \
     postgres:17
```

2 Структура



3 Что хранится в томе



4 Работа с базой данных

```
>_ docker exec -it pg-course psql -U postgres

postgres=# CREATE TABLE categories (
     id SERIAL PRIMARY KEY,
     name  VARCHAR(100)  NOT NULL
     );
INSERT INTO categories (name) VALUES
  ('Электроника'),
  ('Книги');
SELECT * FROM categories;
```

id	name
1	Электроника
2	Книги

5 Пересоздание контейнера без потери данных

Удаляем контейнер
(том остается)

```
>_ docker rm -f pg-course
```

Создаем контейнер заново
с тем же томом

```
>_ docker run -d --name pg-course ...
```

Данные остались
Таблица categories и записи
доступны после пересоздания контейнера

6 Итог

Docker Volume хранит данные независимо от жизненного цикла контейнера.

Важно

- При удалении контейнера том не удаляется.
- Том можно использовать с новым контейнером.
- Для резервного копирования тома используйте `docker run --rm -v ...`

Создаём том PostgreSQL

BASH

```
sudo docker volume create pgdata-course
```

Что важно

- Том pgdata-course создаётся отдельно от контейнера.
- Он будет использоваться для файлов PostgreSQL.
- Проверьте docker volume ls.

Запускаем PostgreSQL 17

BASH

```
sudo docker run -d --name pg-course --mount \
  type=volume,src=pgdata-course,\
  dst=/var/lib/postgresql/data -e \
  POSTGRES_USER=student -e \
  POSTGRES_PASSWORD=Lab5-ChangeMe-2026 -e \
  POSTGRES_DB=shop postgres:17
```

Что важно

- Переменные POSTGRES_* используются для первичной инициализации пустого каталога данных.
- Пароль приведён только для изолированной учебной среды.
- Порт 5432 намеренно не публикуется на хосте; доступ к psql получим через docker exec.

Проверяем готовность PostgreSQL

BASH

```
sudo docker logs pg-course
sudo docker exec pg-course pg_isready -U \
 student -d shop
```

Что важно

- Во время первого запуска инициализация занимает некоторое время.
- Ожидаемый результат pg_isready: accepting connections.
- Если проверка выполнена слишком рано, повторите её.

Создаём таблицу через psql

BASH

```
sudo docker exec pg-course psql -U student -d \
  shop -c "CREATE TABLE categories \
  (id INTEGER GENERATED ALWAYS AS IDENTITY \
  PRIMARY KEY, name TEXT NOT NULL);"
```

Что важно

- docker exec запускает psql внутри работающего контейнера.
- Таблица categories создаётся в базе shop.
- Для первого запуска ожидается CREATE TABLE.

Добавляем учебные записи

```bash
sudo docker exec pg-course psql -U student -d shop \
 -c "INSERT INTO categories (name) VALUES \
 ('Electronics'), ('Books');"
```

Что важно

- Команда добавляет две записи.
- В выводе ожидается INSERT 0 2.
- Данные хранятся в томе pgdata-course.

Запрашиваем данные

BASH

```
sudo docker exec pg-course psql -U student -d \
 shop -c "SELECT id, name FROM categories \
 ORDER BY id;"
```

Что важно

- Ожидаемые категории: Electronics и Books.
- Сохраните вывод SELECT до пересоздания контейнера.
- Номера id назначаются автоматически.

Сохранность данных, диагностика и ограничения

Пересоздание контейнера и повторное использование того же тома

Эксперимент с пересозданием PostgreSQL

1	**Шаг 1**	Убедиться, что SELECT показывает две категории.
2	**Шаг 2**	Остановить pg-course командой docker stop.
3	**Шаг 3**	Удалить контейнер docker rm, не удаляя pgdata-course.
4	**Шаг 4**	Создать новый pg-course из postgres:17 с тем же volume.
5	**Шаг 5**	Дождаться готовности и снова выполнить SELECT.
6	**Шаг 6**	Сравнить результаты до удаления и после пересоздания.

Удаляем только контейнер

```bash
sudo docker stop pg-course
sudo docker rm pg-course
sudo docker volume ls
```

Что важно

- Контейнер будет удалён.
- Том pgdata-course по-прежнему должен присутствовать в списке.
- Не запускайте docker volume rm и docker volume prune.

Создаём PostgreSQL повторно с тем же томом

BASH

```
sudo docker run -d --name pg-course --mount \
 type=volume,src=pgdata-course,\
 dst=/var/lib/postgresql/data -e \
 POSTGRES_USER=student -e \
 POSTGRES_PASSWORD=Lab5-ChangeMe-2026 -e \
 POSTGRES_DB=shop postgres:17
```

Что важно

- Важно повторно использовать то же имя тома и ту же основную версию PostgreSQL.
- При наличии существующих данных POSTGRES_* не переинициализируют базу.
- Проверьте готовность через pg_isready.

Проверка после пересоздания

```bash
sudo docker exec pg-course pg_isready -U student \
 -d shop

sudo docker exec pg-course psql -U student -d \
 shop -c "SELECT id, name FROM categories \
 ORDER BY id;"
```

Что важно

- Данные должны сохраниться: Electronics и Books.
- Убедитесь, что ID записей не изменились.
- Сохранность продемонстрирована для тома; это не заменяет резервное копирование.

Пересоздание контейнера без потери данных

Как удалить и заново создать Docker-контейнер, сохранив данные в Docker Volume

1 Контейнер работает



Таблица categories уже содержит данные.

2 Остановка контейнера




```
>_ docker stop
   pg-course
```

Контейнер остановлен. Данные остаются в томе.

3 Удаление контейнера




```
>_ docker rm
   pg-course
```

Контейнер удалён.

4 Docker Volume остаётся



Том НЕ удалён. Данные сохранены на диске Docker.

5 Создание нового контейнера




```
>_ docker run -d --name pg-course \
   -e POSTGRES_PASSWORD=secret \
   -v pgdata-course:/var/lib/
   postgresql/data \
   postgres:17
```

Создан новый контейнер, подключён к тому же volume с данными.

6 Проверка данных

```
>_ docker exec -it
   pg-course psql -U postgres
```

```
postgres=# SELECT * FROM
categories;
```

id	name
1	Электроника
2	Книги

Данные сохранились.

Почему данные не потерялись?

- Данные хранятся не в writable layer контейнера.
- Контейнер использует внешний Docker Volume.
- `docker rm` удаляет контейнер, но не том.
- Новый контейнер можно подключить к тому же volume.

Docker Volume (постоянное хранилище)



Данные хранятся вне контейнера и сохраняются при удалении и пересоздании контейнера.

Что важно помнить

- **Без volume** данные обычно теряются после удаления контейнера.
- Удаление тома выполняется отдельно (например, `docker volume rm`).
- Volume подходит для БД и постоянных данных.
- Для резервного копирования нужен backup, не только volume.

Главная идея: контейнер можно пересоздать, а Docker Volume продолжает хранить данные.

Типовые ошибки при работе с томами

- Путь тома указан неправильно — данные могут записаться в другое место.
- При новом пустом томе PostgreSQL создаст новый кластер без прежних таблиц.
- Пароль, заданный при запуске, не изменяет пароль в уже существующем кластере.
- docker volume rm завершится ошибкой, если том используется контейнером.
- Отсутствие свободного места на диске хоста может привести к отказу записи.

Не путайте volume с резервной копией

- Том обеспечивает постоянство хранения при пересоздании контейнера.
- Удаление тома приводит к утрате его данных, если нет копии.
- Удаление таблицы через SQL тоже изменяет данные внутри тома.
- Резервные копии и восстановление — отдельная задача администратора.

Как проверить хранилище

Вопрос	Команда
Существует ли том?	docker volume ls
Куда он подключён?	docker inspect pg-course
Где расположен том?	docker volume inspect pgdata-course
Работает ли PostgreSQL?	docker exec pg-course pg_isready -U student -d shop
Сохранились ли записи?	docker exec pg-course psql -U student -d shop -c "SELECT ..."

Итоги лекции

1. Изменения в записываемом слое исчезают после удаления контейнера.
2. Docker Volume хранит данные независимо от жизненного цикла контейнера.
3. Bind Mount связывает контейнер с путём на хосте и удобен при разработке.
4. Для postgres:17 монтируем том в /var/lib/postgresql/data.
5. При пересоздании PostgreSQL используем прежний том и ту же основную версию СУБД.
6. Постоянство хранения не заменяет резервное копирование.

Контрольные вопросы

1. 1. Что такое записываемый слой контейнера и когда он исчезает?
2. 2. Чем Docker Volume отличается от Bind Mount?
3. 3. Когда удобнее использовать bind mount, а когда named volume?
4. 4. Что означает type=volume,src=pgdata-course,dst=/var/lib/postgresql/data?
5. 5. Почему мы используем postgres:17, а не подставляем postgres:18?
6. 6. Что происходит с named volume после docker rm?
7. 7. Зачем проверять docker inspect и docker volume inspect?
8. 8. Что демонстрирует повторный SELECT после пересоздания контейнера?
9. 9. Почему постоянный том нельзя считать резервной копией?